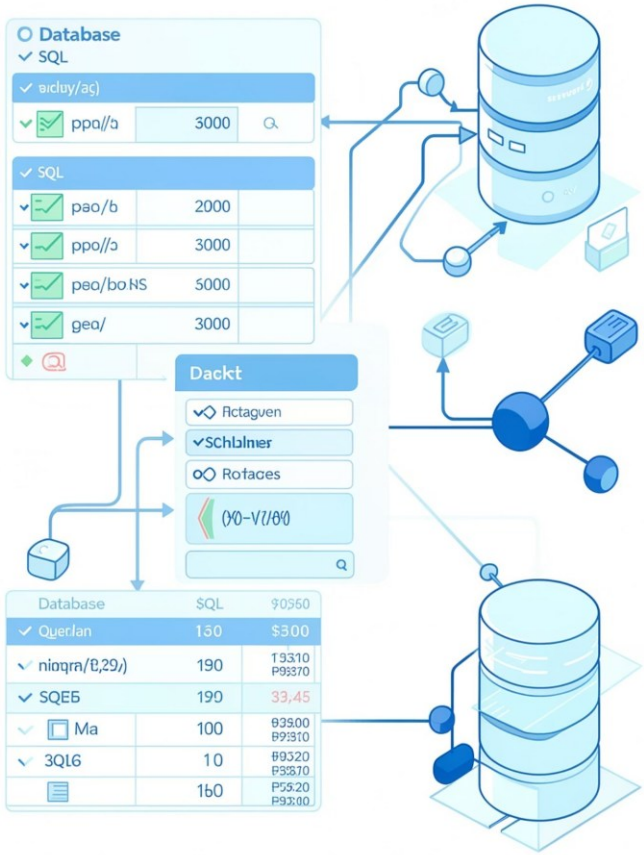
SQL Avanzado - Parte I: Tipos de Datos y Validación Avanzada

Gestión Avanzada de Datos - UTN FRCU

Mg. Andrés Pascal / Mg. Adrián Planas



Temario de la Clase

- Tipos de datos comunes y avanzados (revisión)
- Arrays y su manipulación
- Tipos Enumerados (ENUM)
- Tipos Compuestos y RECORD
- Dominios y Restricciones
- Expresiones Regulares (Pattern Matching)
- JSON y JSONB
- Casting y Constantes avanzadas

Repaso: Tipos Estándar en PostgreSQL

Numéricos

- INTEGER
- BIGINT
- SMALLINT
- NUMERIC(p,s)
- REAL
- MONEY

Alfanuméricos

- VARCHAR(n)
- CHAR(n)
- TEXT

Fechas

- DATE
- TIME
- TIMESTAMP
- INTERVAL

Booleanos

- BOOLEAN

Clave: PostgreSQL permite conversión explícita con `::` (ej: `'2023-01-01'::DATE`) o `CAST('123' AS integer)`

Arrays (Arreglos)

- Definición: Tipo de dato que almacena colecciones dinámicas.
- Declaración: `nombre_columna tipo_dato[]` o `tipo_dato[][]` (multidimensional).
- Constantes: `ARRAY[1,2,3]` o `'{"valor1","valor2"}'::text[]`
- Acceso: `arreglo[1]` (el índice comienza en 1).

Ejemplo de uso en PostgreSQL:

```
CREATE TABLE empleado (
    id SERIAL,
    nombre TEXT,
    habilidades TEXT[]  -- Array de textos
);
```

Arrays - Funciones y Operadores

- `array_length(arreglo, dimension)`: Devuelve la longitud.
- `ANY / ALL`: Buscar dentro del array.

Example code:

```
SELECT * FROM empleado WHERE 'Python' = ANY(habilidades);
SELECT * FROM empleado WHERE 5 > ALL(notas);
```

- `Slicing`: `habilidades[1:2]` (devuelve un sub-array).
- `Concatenación`: `ARRAY[1,2] || ARRAY[3,4] → {1,2,3,4}`.

Tipos Enumerados (ENUM)

Definición: Lista fija de valores predefinidos (ordenados).

Sintaxis:

```
CREATE TYPE estado_pedido AS ENUM ('pendiente', 'en_proceso', 'enviado', 'entregado');
```

Orden: Sigue el orden de definición (pendiente < en_proceso < ...).

Reglas:

- Sensible a mayúsculas/minúsculas.
- No se pueden comparar directamente con otros ENUM (usar CAST).

Uso en tabla:

```
CREATE TABLE pedidos ( id SERIAL, estado estado_pedido );
```

Tipos Compuestos (Composite Types)

Definición: Estructura similar a una "fila" o "registro".

Creación:

```
CREATE TYPE direccion AS (
    calle TEXT,
    numero INT,
    ciudad TEXT
);
```

Construcción: `ROW('Av. Siempreviva', 742, 'Springfield')` o `('valor1', 2, 'valor3')::direccion`.

Acceso: Usando el operador `. -> miembro.calle`

Uso en tabla:

```
CREATE TABLE usuarios (id INT, domicilio direccion);
```

G.A.D.

Tipo RECORD

- Definición: Similar a un compuesto, pero sin estructura fija (implícita).
- Útil para variables en funciones que devuelven filas arbitrarias o cuando se usa FOR ... IN SELECT.
- Declaración: `mi_registro RECORD;`
- Limitación: Solo se puede usar en funciones PL/pgSQL; no como columna de tabla.

Tipo BYTEA (Datos Binarios)

- Definición: Almacena cadenas de bytes (hasta 1GB).
- Uso: Guardar imágenes, archivos PDF, sonidos, ejecutables, etc.
- Inserción: Leer el archivo en format binario desde el cliente e insertarlo directamente con INSERT (o en lugar de leerlo desde el programa cliente se podría hacer un `pg_read_binary_file(), si está en el servidor`).
- Visualización: Las aplicaciones cliente lo traducen al formato nativo (imagen, etc.).

Nota: Para objetos muy grandes, se recomienda Large Objects (`pg_largeobject`), pero BYTEA es más común para archivos pequeños/medianos.

Restricciones CHECK

Definición: Regla declarativa que debe cumplir el valor de una columna o tabla.

Ubicación: En CREATE TABLE o ALTER TABLE.

Ventaja: Son el estándar SQL y más eficientes que los triggers para validaciones simples.

Sintaxis:

```
CREATE TABLE productos (
    id SERIAL,
    precio NUMERIC CHECK (precio > 0),
    fecha_vencimiento DATE CHECK (fecha_vencimiento > CURRENT_DATE)
);
```

CHECK multicolumna:

```
CONSTRAINT ck_fechas CHECK (fecha_egreso IS NULL OR fecha_egreso >= fecha_ingreso)
```

Dominios (CREATE DOMAIN)

Definición: Crear un "alias" de tipo con restricciones CHECK embebidas para reutilizar.

Ventaja: Centraliza reglas de validación (ej: emails, CUIL, códigos postal).

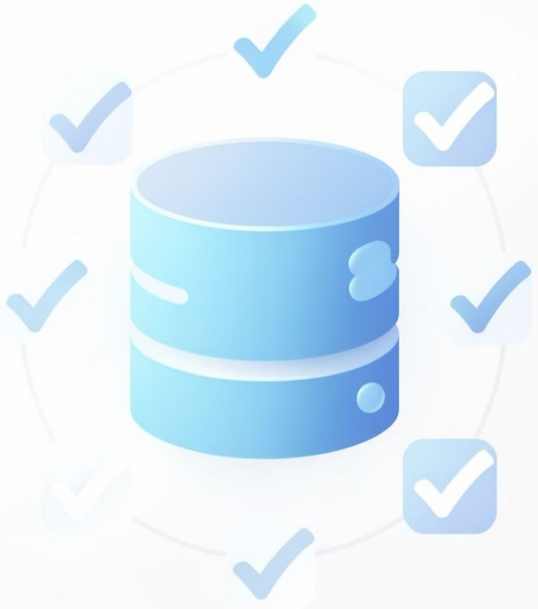
Sintaxis:

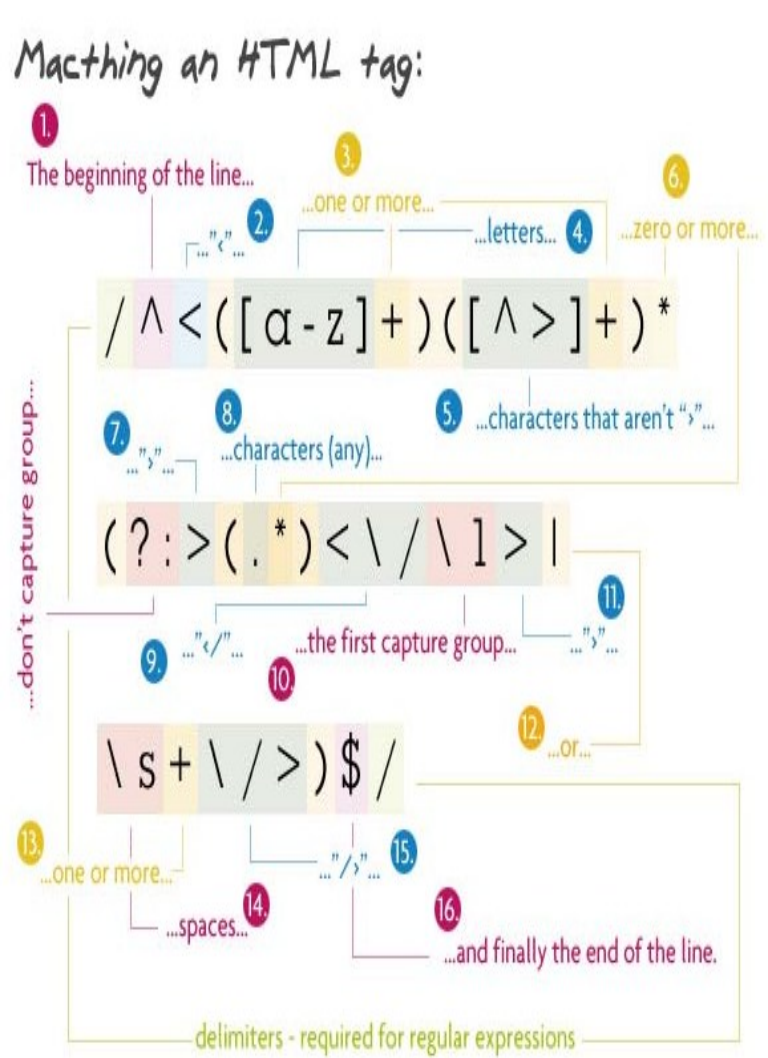
```
CREATE DOMAIN email_valido AS TEXT
CHECK (VALUE ~* '^[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}$');
```

Uso:

```
CREATE TABLE usuarios ( id SERIAL, email email_valido );
```

Ventaja: Si la regla cambia, se modifica el DOMINIO y afecta a todas las tablas automáticamente.





Expresiones Regulares (Operadores)

Definición: Búsqueda de patrones complejos en cadenas de texto (estándar POSIX).

Operadores clave:

- `~` → Coincide con el patrón (Case Sensitive)
- `~*` → Coincide con el patrón (Case Insensitive)
- `!~` → No coincide
- `!~*` → No coincide (Insensitive)

Ejemplo:

```
-- Buscar emails que terminan en .org
SELECT * FROM usuarios WHERE email ~* '\.org$';
```

G.A.D.

Expresiones Regulares - Funciones Principales

- regexp_match(string, pattern, flags): Devuelve el primer match (como texto[]).
- regexp_matches(string, pattern, flags): Devuelve todos los matches (SETOF texto[]).
- regexp_replace(string, pattern, replacement, flags): Reemplaza.
- Flag 'g' = global (todas las ocurrencias).
- regexp_split_to_table(string, pattern): Divide la cadena en filas.
- regexp_split_to_array(string, pattern): Divide la cadena en array.
- Símbolos especiales: . ^ $ * + ? { } [ ] ( ) | \

Ejemplo:

```
SELECT regexp_replace('Hola Mundo', 'o', '0', 'g') → 'H0la Mund0'
SELECT regexp_split_to_table('Esto es un texto de prueba', '\s+'); → divide el texto por espacios
SELECT regexp_matches('Fecha: 18/08/2026 - venc: 31/12/2026','([0-9]{1,2})/([0-9]{1,2})/([0-9]{4})',
```

Expresiones Regulares - Casos Prácticos

Validación de CUIT/CUIL (formato XX-XXXXXXXX-X):

```
columna ~ '^[0-9]{2}-[0-9]{8}-[0-9]$'
```

Extraer números de un texto:

```
SELECT (regexp_match('El precio es $1500', '[0-9]+'))[1]; -- Devuelve '1500'
```

Formatear teléfonos (eliminar no numéricos):

```
SELECT regexp_replace('+54 351 123-4567', '[^0-9]', '', 'g'); -- '543511234567'
```

Introducción a JSON y JSONB

JSON (texto puro): Guarda el JSON exactamente como se envía. Respeta espacios y órden.

JSONB (binario): Guarda en formato binario.

Ventaja: Más rápido de consultar, indexable, elimina espacios duplicados.

Regla: Usar JSONB a menos que necesites preservar el orden exacto de las claves.

Creación:

```
CREATE TABLE eventos (
    id SERIAL,
    datos JSONB
);
```


```
{
  "DocumentType": 1,
  "No.": "S-ORD101001",
  "SellToCustNo": "10000",
  "PostingDate": "2023-04-02",
  "Lines": [
    {
      "LineNo": 10000,
      "Type": 2,
      "No": "1996-S",
      "Quantity": 12,
      "UnitPrice": 1397.3
    },
    {
      "LineNo": 20000,
      "Type": 2,
      "No": "1900-S",
      "Quantity": 4,
      "UnitPrice": 192.8
    }
  ]
}
```

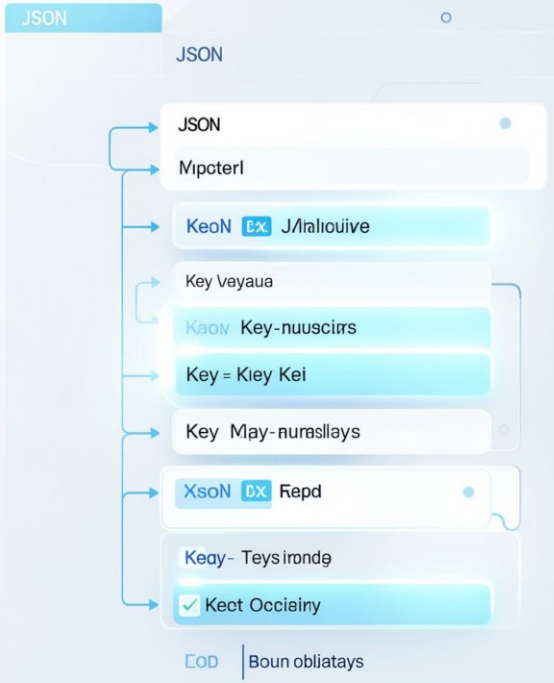
JSON/JSONB - Operadores de Acceso

- `->` : Obtiene un campo como JSON (mantiene el tipo JSON).
- `->>` : Obtiene un campo como Texto (cadena).
- `#>` : Obtiene un objeto anidado (ruta) como JSON.
- `#>>` : Obtiene un objeto anidado (ruta) como Texto.

Ejemplo:

```
SELECT '{"nombre": "Juan", "edad": 35}'::jsonb -> 'nombre';

-- datos: {"cliente": {"nombre": "Ana", "id": 5}, "total": 100}
SELECT datos->'cliente'->>'nombre' FROM eventos;  -- Devuelve 'Ana'
SELECT datos#>'{cliente,id}' FROM eventos;        -- Devuelve 5
```

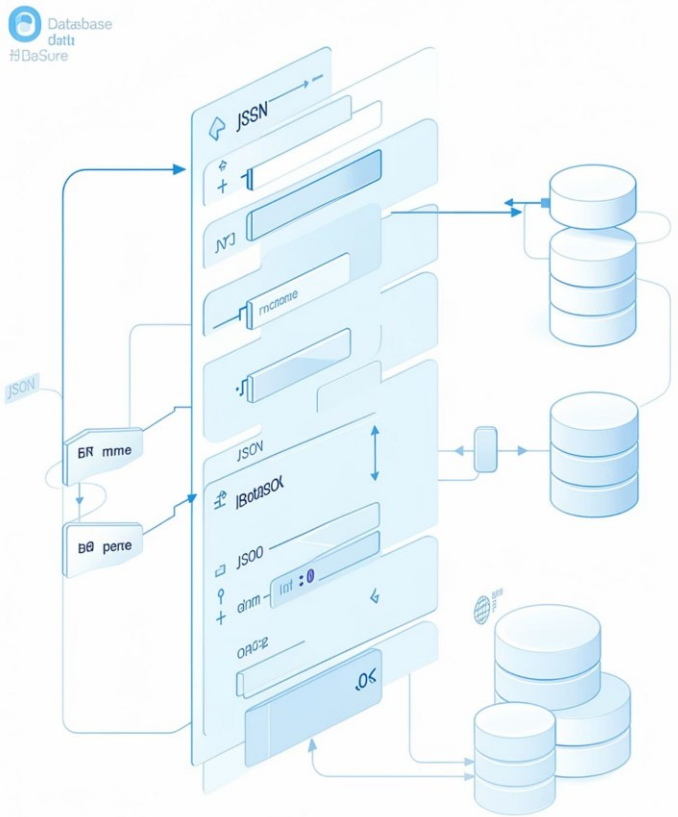


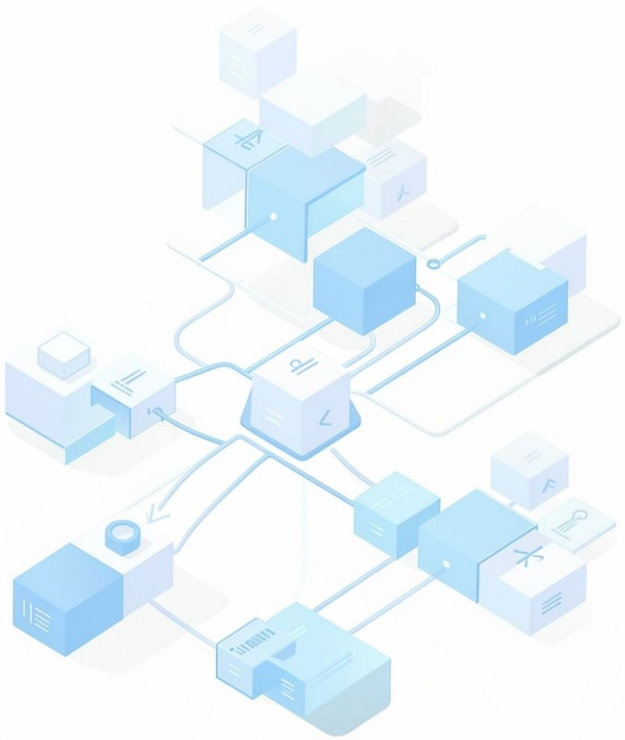
JSON/JSONB - Operadores de Contención e Inclusión

- **@>** : ¿El JSON de la izquierda CONTIENE al de la derecha?
- **<@** : ¿El JSON de la izquierda ESTÁ CONTENIDO en el de la derecha?
- **?** : ¿Existe la clave?
- **?|** : ¿Existe ALGUNA de las claves en el array?
- **?&** : ¿Existen TODAS las claves en el array?

Ejemplo:

```
SELECT * FROM eventos WHERE datos @> '{"estado":
"activo"}';
SELECT * FROM eventos WHERE datos ? 'total';
```





JSON/JSONB - Funciones de Creación y Agregación

- jsonb_build_object('clave1', valor1, 'clave2', valor2): Crea un objeto JSON.
- jsonb_agg(expresion): Agrupa filas en un array JSON.
- jsonb_object_agg(clave, valor): Agrupa en un objeto JSON.

Ejemplo:

```
SELECT jsonb_build_object('id', id, 'nombre', nombre)
FROM usuarios; -- Devuelve: {"id": 1, "nombre": "Carlos"}

SELECT jsonb_object_agg(nombre, precio)FROM productos;
```

JSON/JSONB - Modificación y Actualización

jsonb_set(targeto, ruta, nuevo_valor, crear_si_no_existe):

```
UPDATE eventos SET datos = jsonb_set(datos,
'{cliente,nombre}', '"Pedro"', true);
```

jsonb_insert(targeto, ruta, nuevo_valor, insertar_despues): Inserta en arrays u objetos.

jsonb_delete(targeto, clave) (o usar -):

```
SELECT datos - 'total' FROM eventos; -- Elimina la clave 'total'
SELECT datos #- '{cliente,email}' FROM eventos;
```



Casting y Constantes Tabla (repaso)

Casting:

Convertir un tipo a otro.

```
'123'::INTEGER o CAST('123' AS INTEGER)
```

```
'{"a": 1}'::JSONB
```

Constante Tabla (VALUES):

```
SELECT * FROM (VALUES ('Ana', 30), ('Luis', 25)) AS t(nombre, edad);
```

Útil para pruebas o para crear tablas temporales inline sin CREATE TABLE.

Resumen y Buenas Prácticas



Arrays

Úsalos para listas simples (ej: etiquetas, días de semana).



ENUM

Preferible a códigos numéricos cuando los valores son fijos y pocos.



DOMINIOS y CHECK

SIEMPRE que puedas, evita triggers para validaciones declarativas.



RegEx

Perfecto para validar formatos (CUIL, mail, patentes) y limpieza de datos.



JSONB

El rey de los datos semiestructurados. Ideal para datos variables o integración con APIs.